

PRONTUÁRIO: _____

Data: __/__/____

Horário: 3h00

✓ A prova é individual e sem consulta. ✓ Não é permitido utilizar qualquer código implementado por você anteriormente. ✓ Não coloque seu nome no projeto. ✓ Atribui-se nota zero à prova em desacordo com os itens acima.	✓ A prova deve ser nomeada da seguinte forma: PRONTUARIO_P_SIMULADA, com o "SC". ✓ Não envie apenas as classes, mas todo o projeto. ✓ Envie o projeto como zip (ou apenas os .java, se for só treino pessoal).
--	--

Considere o contexto a seguir:

Você vai implementar o núcleo de um sistema de controle de estacionamento. Você deve implementar as três classes do modelo — Veiculo, Ticket (representa a permanência de um veículo) e Estacionamento (gerencia as vagas e os tickets) —, conforme o diagrama a seguir:

Veiculo
- placa: String - modelo: String
+ Veiculo(placa: String, modelo: String) + getPlaca(): String + getModelo(): String

Nesta versão, a validação do padrão da placa (Mercosul) deve ser feita no próprio construtor de Veiculo, lançando *IllegalArgumentException* se o formato for inválido — veja a regra correspondente abaixo.

Estacionamento	Ticket
- vagasTotais: int - tickets: Ticket[] - valorHora: double	- codigo: String - veiculo: Veiculo - entrada: LocalDateTime - saida: LocalDateTime
+ Estacionamento(vagasTotais: int, valorHora: double) + registrarEntrada(v: Veiculo): Ticket + registrarSaida(placa: String): double + vagasDisponiveis(): int + veiculosNoPatio(): Veiculo[]	+ Ticket(codigo: String, veiculo: Veiculo) + finalizar(): void + estaAberto(): boolean + tempoPermanencia(): Duration + valorCobrado(valorHora: double): double

Relacionamentos: Estacionamento possui de 0 a N objetos Ticket (composição, array, tamanho máximo = vagasTotais). Cada Ticket está associado a exatamente 1 Veiculo (relacionamento unidirecional; Veiculo não conhece Ticket nem Estacionamento).

Regras de negócio:

- A placa de um Veiculo segue o padrão Mercosul: 3 letras maiúsculas, 1 dígito, 1 letra maiúscula, 2 dígitos (ex.: "ABC1D23"). O construtor de Veiculo deve validar esse padrão via expressão regular (`String.matches()`) e lançar *IllegalArgumentException* caso a placa não siga o formato.
- entrada de um Ticket é sempre o momento da criação (`LocalDateTime.now()`).
- registrarEntrada(v) só aceita o veículo se houver vaga disponível (`vagasDisponiveis() > 0`); o código do ticket deve ser gerado concatenando a placa do veículo com um timestamp/sequencial simples, garantindo que seja único.
- finalizar() registra saida como o momento atual.
- tempoPermanencia() retorna a *Duration* entre entrada e saida (ou entre entrada e o momento atual, se o ticket ainda estiver aberto) — utilize a API *Date/Time* (`Duration.between()`), nunca cálculo manual.

- valorCobrado(valorHora) cobra proporcionalmente às horas de permanência, sempre arredondando a fração de hora para cima (ex.: 1h05min conta como 2 horas). Dica: use os métodos de Duration (toMinutes(), etc.) para o arredondamento.
- registrarSaida(placa), no Estacionamento, localiza o ticket aberto correspondente à placa, chama finalizar(), calcula e retorna o valor cobrado (usando o valorHora do estacionamento).
- veiculosNoPatio() retorna os veículos cujo ticket ainda está aberto (não finalizado).
- vagasDisponiveis() = vagasTotais menos a quantidade de tickets atualmente abertos.

Execute as atividades a seguir para a implementação do exercício em Java. Para a atribuição da nota será levada em conta não apenas a funcionalidade, mas a qualidade, adequação e pertinência de cada solução.

#	Descrição	Pont.
1	Implemente a classe Veiculo, validando o formato da placa via regex no construtor.	1,0pt
2	Crie as demais classes do modelo (Estacionamento e Ticket) com os modificadores de acesso adequados ao encapsulamento.	0,5pt
3	Estruture a associação de composição entre Estacionamento e Ticket usando array de tamanho vagasTotais.	0,5pt
4	Implemente o construtor de Ticket e o método finalizar()/estaAberto().	1,0pt
5	Implemente tempoPermanencia() usando Duration, cobrindo corretamente os dois casos (ticket aberto e fechado).	1,5pt
6	Implemente valorCobrado(valorHora), com o arredondamento de hora para cima calculado a partir da Duration (sem cálculo manual de milissegundos).	1,5pt
7	Implemente registrarEntrada(v) no Estacionamento, verificando vaga disponível e gerando um código de ticket único.	1,0pt
8	Implemente registrarSaida(placa), localizando o ticket certo, finalizando-o e retornando o valor cobrado.	1,0pt
9	Implemente vagasDisponiveis() e veiculosNoPatio().	0,5pt
10	Crie uma classe Principal com um método main() que crie um Estacionamento com poucas vagas (ex.: 2), tente registrar mais entradas do que vagas disponíveis (tratando o erro), registre ao menos uma saída e imprima o valor cobrado e as vagas disponíveis ao final.	1,0pt